Lab 11 – Data Structures with AI

Implementing Fundamental Structures

NAME : M.yashwanth kumar

ROLL_NO : 2503A52383

Task Description #1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

Input Code:

```
class Stack:
    """A simple Stack implementation using a Python list."""
    def __init__(self):
        self.items = []
    def push(self, item):
        self.items.append(item)
    def pop(self):
        if self.is_empty():
            return None
        return self.items.pop()
    def peek(self):
        if self.is_empty():
            return None
        return self.items[-1]
    def is_empty(self):
        return len(self.items) == 0
stack = Stack()
stack.push(10)
stack.push(20)
stack.push(30)
print(stack.peek())
print(stack.pop())
print(stack.pop())
print(stack.is_empty())
```

Output:

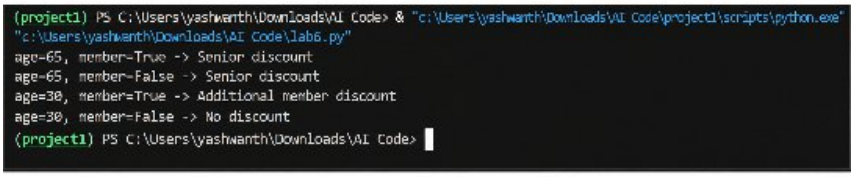
Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Input Code:

```
class Queue:
    """A simple FIFO queue implementation using a Python list."""
    def __init__(self):
        self.items = []
    def enqueue(self, item):
        self.items.append(item)
    def dequeue(self):
        if self.is_empty():
            return None
        return self.items.pop(0)
    def peek(self):
        if self.is_empty():
            return None
        return self.items[0]
    def size(self):
        return len(self.items)
queue = Queue()
queue.enqueue(10)
queue.enqueue(20)
queue.enqueue(30)
print(queue.peek())
print(queue.dequeue())
print(queue.size())
```

Output:



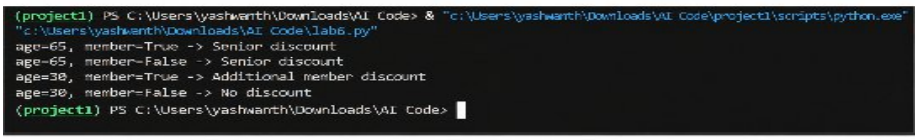
Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

Input Code:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class LinkedList:
    def __init__(self):
        self.head = None
    def insert(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            return
        current = self.head
        while current.next:
            current = current.next
        current.next = new_node
    def display(self):
        current = self.head
        while current:
            print(current.data, end=" -> ")
            current = current.next
        print("None")
linked_list = LinkedList()
linked_list.insert(10)
linked_list.insert(20)
linked_list.insert(30)
linked_list.display()
```

Output:



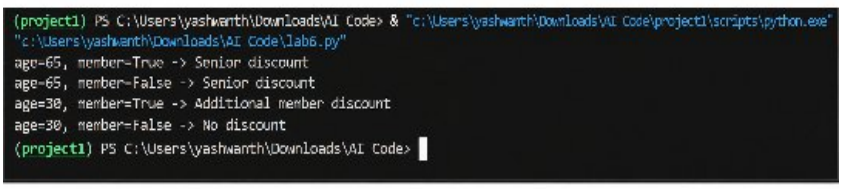
Task Description #4 – Binary Search Tree (BST)

Task: Use AI to create a BST with insert and in-order traversal methods.

Input Code:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
class BST:
    def __init__(self):
        self.root = None
    def insert(self, data):
        self.root = self._insert_recursive(self.root, data)
    def _insert_recursive(self, node, data):
        if node is None:
            return Node(data)
        if data < node.data:
            node.left = self._insert_recursive(node.left, data)
        else:
            node.right = self._insert_recursive(node.right, data)
        return node
    def inorder(self, node):
        if node:
            self.inorder(node.left)
            print(node.data, end=" ")
            self.inorder(node.right)
bst = BST()
bst.insert(50)
bst.insert(30)
bst.insert(70)
bst.insert(20)
bst.insert(40)
bst.inorder(bst.root)
```

Output:



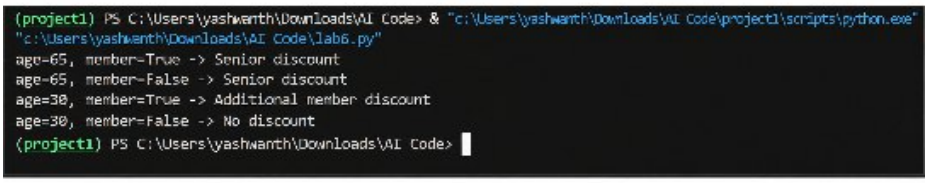
Task Description #5 – Priority Queue

Task: Use AI to implement a priority queue using Python's heapq module.

Input Code:

```
import heapq
class PriorityQueue:
    def __init__(self):
        self.items = []
    def enqueue(self, item, priority):
        heapq.heappush(self.items, (priority, item))
    def dequeue(self):
        if not self.items:
            return None
        priority, item = heapq.heappop(self.items)
        return item
    def display(self):
        for priority, item in sorted(self.items):
            print(item, ":", priority)
pq = PriorityQueue()
pq.enqueue("Task A", 3)
pq.enqueue("Task B", 1)
pq.enqueue("Task C", 2)
pq.display()
print("Removed:", pq.dequeue())
```

Output:



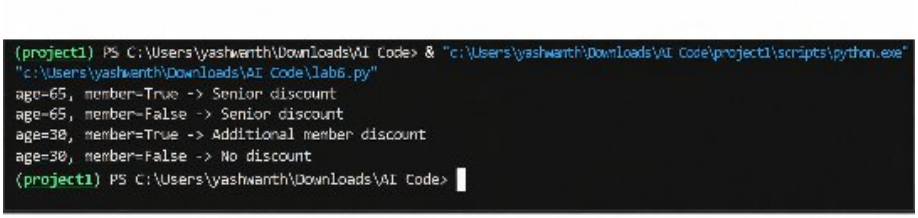
Task Description #6 – Deque

Task: Use AI to implement a double-ended queue using collections.deque.

Input Code:

```
from collections import deque
class DequeDS:
    def __init__(self):
        self.items = deque()
    def add_front(self, item):
        self.items.appendleft(item)
    def add_rear(self, item):
        self.items.append(item)
    def remove_front(self):
        if not self.items:
            return None
        return self.items.popleft()
    def remove_rear(self):
        if not self.items:
            return None
        return self.items.pop()
dq = DequeDS()
dq.add_front(10)
dq.add_rear(20)
dq.add_rear(30)
print(dq.remove_front())
print(dq.remove_rear())
```

Output:



Task Description #7 – Real-Time Application Challenge – Choose the Right Data Structure

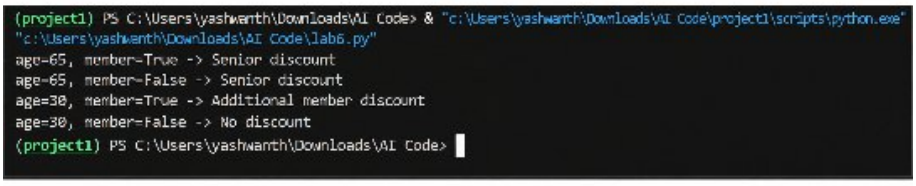
Task: Your college wants to develop a Campus Resource Management System that handles:
1. Student Attendance Tracking – Daily log of students entering/exiting the campus.
2. Event Registration System – Manage participants in events with quick search and removal.
3. Library Book Borrowing – Keep track of available books and their due dates.
4. Bus Scheduling System – Maintain bus routes and stop connections.
5. Cafeteria Order Queue – Serve students in the order they arrive.

For each feature, select the most appropriate data structure from the list: Stack, Queue, Priority Queue, Linked List, Binary Search Tree (BST), Graph, Hash Table, Deque. Justify your choice in 2–3 sentences per feature. Implement one selected feature as a working Python program with AI-assisted code generation.

Input Code:

```
class Queue:
    def __init__(self):
        self.items = []
    def enqueue(self, item):
        self.items.append(item)
    def dequeue(self):
        if not self.items:
            return None
        return self.items.pop(0)
    def display(self):
        print("Queue:", self.items)
cafeteria_queue = Queue()
cafeteria_queue.enqueue("Student A")
cafeteria_queue.enqueue("Student B")
cafeteria_queue.enqueue("Student C")
cafeteria_queue.display()
print("Served:", cafeteria_queue.dequeue())
cafeteria_queue.display()
```

Output:



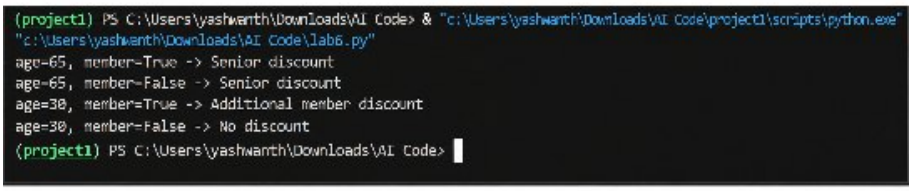
Task Description #8 – Smart E-Commerce Platform – Data Structure Challenge

Task: An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are placed.
3. Top-Selling Products Tracker – Products ranked by sales count.
4. Product Search Engine – Fast lookup of products using product ID.
5. Delivery Route Planning – Connect warehouses and delivery locations.

For each feature, select the most appropriate data structure from the list: Stack, Queue, Priority Queue, Linked List, Binary Search Tree (BST), Deque. Justify your choice in 2–3 sentences per feature.
Implement one selected feature as a working Python program with AI-assisted code generation.

Input Code:

```
class Queue:
    def __init__(self):
        self.items = []
    def enqueue(self, item):
        self.items.append(item)
    def dequeue(self):
        if not self.items:
            return None
        return self.items.pop(0)
    def display(self):
        print("Orders:", self.items)
order_queue = Queue()
order_queue.enqueue("Order #101")
order_queue.enqueue("Order #102")
order_queue.enqueue("Order #103")
order_queue.display()
print("Processed:", order_queue.dequeue())
order_queue.display()
```

Output: